perplexity

Can you also generate an content. Document like a— Keep it concise. But the technical definitions and all, and key architectural decisions or thought process should be replicated. As. It is, um. Okay. Uh. Keep the format with minimal tokenization. Words. I mean, with minimum words what you can convey the most, uh, it's for. On that. What I want you to include in that context document is. For, uh, I want to put it into my LLM, uh. That's the use case. First of all, and what. I want you to do is, uh, uh, the. Whole AI Conix architecture. Like, uh, the. Whole— AI Conix architecture I want you, uh, from phase 1 to phase 2. Keep phase 1 a little short, uh. The, like, whole process of using different. AWS services like, uh, Data, Glue, Databrew, um. Kind of thing, and, uh, but keep the phase 2 a bit detailed along with all the target features that we have, the one which we are keeping with referenced to the, uh, uh, sober AI and all. Right, uh. That being said, um. Yeah, plus. The. Main thing is the SageMaker Unified Studio thing, the. Also. Add. Add in the progress of our progress that what we have, uh, uh. Thought. Of, like, uh, as a 2.1 phase, like what, uh, we are doing

Below is a **compact context document** you can paste into your LLM. I kept Phase 1 short, Phase 2 more detailed, and included the Phase 2.1 direction. It is written as a single architecture note with minimal wording but preserved decisions and definitions. [1] [2] [3]

AI ConneX Context

Goal

Build an industrial ML platform on AWS. Phase 1 prepares data. Phase 2 trains, evaluates, registers, and deploys models. Primary initial use cases: regression and anomaly detection for industrial data. Core training dataset: CMAPSS-style engine time-series. [4] [5] [6]

Phase 1

Phase 1 uses AWS Glue, AWS Glue DataBrew, and SageMaker Data Wrangler as data prep tools. Glue handles large-scale ETL and cataloging. DataBrew handles visual cleaning and recipe-based transforms. Data Wrangler handles SageMaker-native exploration and feature engineering. Phase 1 outputs cleaned, versioned datasets and metadata to S3. [7] [8] [9]

Phase 2

Phase 2 is the ML build layer. It reads cleaned S3 datasets, validates schema, discovers features, splits data, preprocesses, trains, evaluates, registers the model, and prepares deployment. Use SageMaker Unified Studio as the main workspace because it unifies data, notebooks, ML, governance, and workflow execution. [3] [1]

Phase 2.1

Phase 2.1 is local-first development. Write notebooks and Python scripts locally in Jupyter/CLI/laptop/Colab for preprocessing, training, evaluation, and model export. Run and debug locally first. Then convert the same scripts into SageMaker Processing jobs, Training jobs, and Pipeline steps. [10] [11] [12]

Dataset

CMAPSS is a multivariate engine time-series dataset for predictive maintenance and remaining useful life (RUL) regression. It contains unit/cycle data, operating settings, and sensor measurements. It is sequence-based, so splits must avoid leakage across engines and time. [5] [6] [4]

S3 layout

Current example:

- `s3://ai-connex-s3-cleaned-data/cmapss/v1/clean_train.parquet`
- `s3://ai-connex-s3-cleaned-data/cmapss/v1/feature_dictionary.csv`
- `s3://ai-connex-s3-cleaned-data/cmapss/v1/metadata/`
  This is the Phase 2 input contract. [13]

Pipeline principle

One generic pipeline framework, many branches. The pipeline decides path from metadata plus dataset inspection. Key routing fields:

- `task_type`
- `data_type`
- `sequence_flag`
- `deployment_mode`
  These may be prewritten or inferred in the preprocessing job and stored back with the dataset as metadata. [14] [15] [16]

Generic flow

S3 cleaned data → dataset loader → schema validation → feature discovery → train/validation split → preprocessing → training job → evaluation → model registry → deployment. [17] [18] [19]

Dynamic branching logic

- `task_type = regression` → supervised regression path.
- `task_type = anomaly` → anomaly path.
- `sequence_flag = true` → chronological or unit-aware split.
- `sequence_flag = false` → standard random or stratified split.
- `deployment_mode = batch` → batch transform.
- `deployment_mode = realtime` → endpoint deployment. [19] [20] [21]

Regression branch

Use when target is numeric, e.g. RUL. Typical steps:

- keep valid rows,
- cap or remove obvious outliers,
- scale/encode as needed,
- train regressors such as XGBoost or Linear Learner,
- evaluate with RMSE, MAE, R^2. [6] [22] [5]

Anomaly branch

Use when rare events or abnormal behavior are the target. Typical steps:

- preserve outliers,
- train on normal-heavy data,
- set contamination/threshold,
- use anomaly models such as Random Cut Forest or isolation-based methods,

- evaluate with precision@k, recall, false alert rate, score drift. [23] [24] [6]

Orchestration

Use SageMaker Pipelines as the main DAG engine. Use ProcessingStep for load/validate/preprocess/evaluate. Use TrainingStep for model fit. Use TuningStep only when optimization is needed. Use Model Registry for versioning and approval. Deployment happens after approval. [18] [25] [17]

Decision rule

The pipeline should not be fixed for every dataset. It should infer or read metadata, then select the right branch and job sequence. Preprocessing can also write the dataset context file if the metadata is missing. [15] [26] [27]

Target architecture philosophy

- Phase 1: clean and standardize data.
- Phase 2: make the data trainable and build models.
- Phase 2.1: build locally first, then move identical logic into SageMaker jobs.
- Keep the first model path narrow and working before adding extra algorithms, HPO, monitoring, or deployment complexity. [28] [10] [17]

If you want, I can next turn this into a **clean `.md` context file** with tighter wording and section headers optimized for LLM ingestion.

⁂

1. https://aws.amazon.com/sagemaker/unified-studio/
2. https://docs.aws.amazon.com/solutions/collaborative-unified-data-and-ai-development-on-aws/
3. https://docs.aws.amazon.com/sagemaker-unified-studio/latest/userguide/sagemaker-unified-studio-user-guide.pdf
4. https://data.nasa.gov/dataset/cmapss-jet-engine-simulated-data
5. https://github.com/makinarocks/awesome-industrial-machine-datasets/blob/master/data-explanation/C-MAPSS/README.md
6. ml_pipelines_regression_vs_anomaly.html
7. https://www.mindmeshacademy.com/certifications/aws/aws-certified-machine-learning-engineer-associate/study-guide/2-2-4-aws-transformation-tools-glue-databrew-emr-and-data-wrangler
8. https://aws.amazon.com/blogs/big-data/prepare-transform-and-orchestrate-your-data-using-aws-glue-databrew-aws-glue-etl-and-aws-step-functions/
9. https://www.youtube.com/watch?v=XDnLRP_9q1A
10. https://docs.aws.amazon.com/sagemaker/latest/dg/pipelines-local-mode.html
11. https://docs.aws.amazon.com/sagemaker/latest/dg/processing-job.html
12. https://docs.aws.amazon.com/sagemaker/latest/dg/train-remote-decorator.html
13. sorba_vs_ai_connex_feature_parity.md

14. https://docs.aws.amazon.com/sagemaker/latest/dg/define-pipeline.html
15. https://docs.aws.amazon.com/sagemaker/latest/dg/build-and-manage-steps.html
16. https://sagemaker-examples.readthedocs.io/en/latest/sagemaker-pipeline-parameterization/parameterized-pipeline.html
17. https://docs.aws.amazon.com/sagemaker/latest/dg/pipelines-overview.html
18. https://docs.aws.amazon.com/sagemaker/latest/dg/model-registry.html
19. https://docs.aws.amazon.com/sagemaker/latest/dg/model-registry-deploy.html
20. https://docs.aws.amazon.com/sagemaker/latest/dg/algorithms-choose.html
21. https://aws.amazon.com/blogs/machine-learning/create-sagemaker-pipelines-for-training-consuming-and-monitoring-your-batch-use-cases/
22. https://docs.aws.amazon.com/sagemaker/latest/dg/algos.html
23. https://aws.amazon.com/blogs/machine-learning/use-the-built-in-amazon-sagemaker-random-cut-forest-algorithm-for-anomaly-detection/
24. https://arxiv.org/html/2410.14579v1
25. https://docs.aws.amazon.com/sagemaker/latest/dg/build-and-manage-steps-types.html
26. https://github.com/aws/sagemaker-python-sdk/issues/3169
27. https://aws.amazon.com/blogs/machine-learning/best-practices-and-design-patterns-for-building-machine-learning-workflows-with-amazon-sagemaker-pipelines/
28. SageMaker_Automation_Methods_Guide-1-.docx.pdf
29. https://d1.awsstatic.com/onedam/marketing-channels/website/aws/en_US/solutions/approved/documents/architecture-diagrams/collaborative-unified-data-and-ai-development-on-aws.pdf
30. https://d1.awsstatic.com/solutions/guidance/architecture-diagrams/collaborative-unified-data-and-ai-development-on-aws.pdf
31. https://pages.awscloud.com/rs/112-TZM-766/images/DAT203-English.pdf
32. https://dev.to/aws-builders/architecting-your-genai-data-pipeline-with-aws-native-services-2nnm
33. https://zenn.dev/kiiwami/articles/e10e72fc74aeedbe?locale=en
34. https://aws.amazon.com/pt/sagemaker/unified-studio/
35. https://aws-experience.com/amer/smb/e/f1600/learning-series--data-processing-in-next-generation-amazon-sagemaker